

Лабораторная №9. Работа с логическим анализатором

Цель работы

Изучить принципы работы логического анализатора, освоить управление GPIO через API `libgpiod 2.x` и получить практические навыки измерения цифровых сигналов и декодирования UART в Sigrok/PulseView.

Оборудование и предварительные требования

- одноплатный компьютер Lichee RV Dock с ALT Linux;
- USB-логический анализатор, поддерживаемый драйвером `fx2lafw`;
- соединительные провода;
- компьютер с Sigrok и PulseView;
- компилятор GCC, библиотека `libgpiod 2.x` и заголовочные файлы;
- доступ к UART TX и выводу PE16 платы.

Опасность повреждения

Собирайте и изменяйте схему только при выключенном питании. Соедините GND платы и анализатора, подключайте к GPIO только входной канал анализатора и не подавайте 5 В на выводы Lichee RV Dock.

Ключевые понятия

Определение

Логический анализатор — прибор, который дискретизирует логические уровни нескольких цифровых линий и записывает их изменения во времени.

Логический анализатор удобен для длительного многоканального захвата и декодирования цифровых протоколов. Осциллограф показывает фактическую форму и амплитуду сигнала, поэтому лучше подходит для анализа фронтов, выбросов, шумов и нарушений электрических уровней.

В ALT Linux доступны пакеты проекта Sigrok:

- `sigrok-cli` — утилита командной строки;
- `pulseview` — графический интерфейс.

Определение

Sigrok — проект программного обеспечения для работы с измерительными приборами и декодирования протоколов.

PulseView является графическим приложением проекта Sigrok для настройки захвата, отображения сигналов и работы с декодерами.

Определение

GPIO-линия — отдельный управляемый цифровой сигнал, идентифицируемый GPIO-чипом и смещением линии внутри него.

Порядок выполнения

Установка программного обеспечения

Установите программы для анализатора:

```
sudo apt-get install sigrok-cli sigrok-firmware-fx2lafw pulseview
```

Подключите анализатор к компьютеру и проверьте обнаружение:

```
sigrok-cli --scan
```

Проверка

Ожидаемый результат. В списке устройств присутствует анализатор с драйвером fx2lafw. Если устройство не найдено, проверьте USB-подключение, пакет прошивки и системный журнал.

Измерение цифрового импульса

При выключенном питании соедините GND анализатора с GND платы, а один входной канал CH0 или CH1 — с исследуемым выводом GPIO. После проверки схемы включите питание и запустите PulseView:

```
pulseview
```

Выберите анализатор fx2lafw, оставьте нужный канал, задайте частоту дискретизации и запустите захват. Для тестового сигнала с длительностью импульса 10 мс измерьте длительности высокого и низкого уровней, период и частоту.

Проверка

Проверка настройки. На один период должно приходиться достаточно отсчётов для устойчивого измерения. Если фронты отображаются нестабильно или пропускаются, увеличьте частоту дискретизации.

Декодирование UART

UART-консоль Lichee RV Dock передаёт загрузочные сообщения через TX. При выключенном питании подключите:

- GND анализатора к GND платы;
- CH0 анализатора к TX платы.

В PulseView выполните следующие действия:

1. Оставьте подключённый канал.
2. Установите частоту дискретизации 1 МГц или выше.
3. Добавьте декодер UART.
4. Назначьте вход декодера каналу, подключённому к TX.
5. Установите параметры формата 115200 8N1.

```
Baud rate: 115200
Data bits: 8
Parity: none
Stop bits: 1
Bit order: lsb-first
Data format: ascii
Invert RX: no
Invert TX: no
Sample point (%): 50
```

Проверка

Ожидаемый результат. Декодер отображает читаемые фрагменты загрузочного журнала. Если символы искажены, сначала проверьте общий GND, выбранный канал, скорость 115200 бод и отсутствие инверсии.

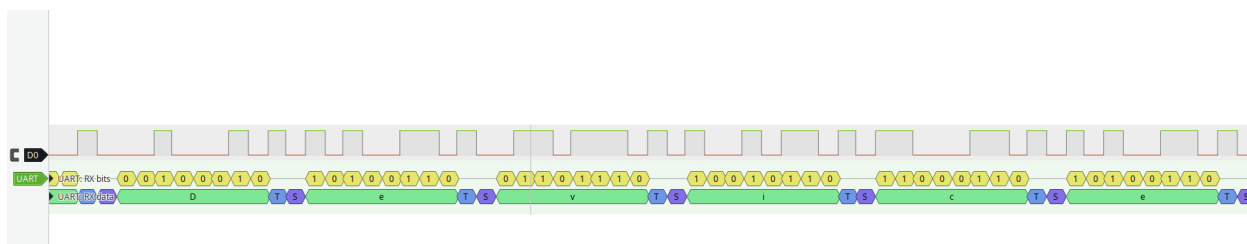


Рис. 1 – Декодирование загрузочного вывода UART 115200 8N1 в PulseView

Работа с libgpiod 2.x

Определение

libgpiod — библиотека пользовательского пространства для работы с символьным интерфейсом GPIO ядра Linux.

В ALT Linux пакет `libgpiod2` содержит библиотеку времени выполнения, а `libgpiod-devel` — заголовки и файлы для компоновки программ:

```
sudo apt-get install libgpiod2 libgpiod-devel
```

Проверьте установленную версию и доступные GPIO-чипы:

```
gpiodetect --version
gpiodetect
gpioinfo
```

Внимание

Синтаксис утилит и C API libgpiod 1.x и 2.x различается. Команды и код этой работы рассчитаны на libgpiod 2.x.

Основные объекты API 2.x:

- `struct gpiod_chip` представляет открытое символьное устройство `/dev/gpiochipN`;
- `struct gpiod_line_settings` хранит направление и начальное значение линии;
- `struct gpiod_line_config` связывает настройки со смещениями линий;
- `struct gpiod_request_config` хранит параметры запроса, включая имя потребителя;
- `struct gpiod_line_request` представляет полученный исключительный доступ к линиям.

Основные функции, используемые в каркасе программы:

```
#include <gpiod.h>

struct gpiod_chip *gpiod_chip_open(const char *path);
void gpiod_chip_close(struct gpiod_chip *chip);

struct gpiod_line_settings *gpiod_line_settings_new(void);
int gpiod_line_settings_set_direction(
    struct gpiod_line_settings *settings,
    enum gpiod_line_direction direction
);
int gpiod_line_settings_set_output_value(
    struct gpiod_line_settings *settings,
    enum gpiod_line_value value
);
void gpiod_line_settings_free(struct gpiod_line_settings *settings);

struct gpiod_line_config *gpiod_line_config_new(void);
int gpiod_line_config_add_line_settings(
    struct gpiod_line_config *config,
    const unsigned int *offsets,
    size_t num_offsets,
    struct gpiod_line_settings *settings
);
void gpiod_line_config_free(struct gpiod_line_config *config);

struct gpiod_request_config *gpiod_request_config_new(void);
void gpiod_request_config_set_consumer(
    struct gpiod_request_config *config,
```

```

    const char *consumer
);
void gpiod_request_config_free(struct gpiod_request_config *config);

struct gpiod_line_request *gpiod_chip_request_lines(
    struct gpiod_chip *chip,
    struct gpiod_request_config *req_config,
    struct gpiod_line_config *line_config
);
int gpiod_line_request_set_value(
    struct gpiod_line_request *request,
    unsigned int offset,
    enum gpiod_line_value value
);
void gpiod_line_request_release(struct gpiod_line_request *request);

```

В API 2.x для логических состояний применяются `GPIO_LINE_VALUE_INACTIVE` и `GPIO_LINE_VALUE_ACTIVE`, а не необозначенные целые значения.

Определение линии PE16

В используемой конфигурации PE16 соответствует смещению 144 в `/dev/gpiochip0`. Перед работой подтвердите это соответствие и убедитесь, что линия свободна:

```

gpioinfo -c 0 144
gpioget -c 0 144
gpiomon -c 0 144

```

Для ручной установки выхода утилитой `gpioset` используется синтаксис 2.x:

```

gpioset -c 0 144=1

```

Внимание

Номер `/dev/gpiochip0` и смещение 144 необходимо проверять на фактически загруженной системе. Device Tree и версия ядра могут изменить представление GPIO; занятая линия не должна запрашиваться программой.

Каркас генератора меандра

Приведённый каркас открывает GPIO-чип, настраивает PE16 как выход и освобождает ресурсы после `Ctrl+C`. Самостоятельно дополните цикл формированием высокого и низкого уровней и обработкой длительностей из аргументов командной строки.

```

#include <gpiod.h>
#include <signal.h>

```

```

#include <stdio.h>

#define GPIO_CHIP "/dev/gpiochip0"
#define GPIO_OFFSET 144

static volatile sig_atomic_t keep_running = 1;

static void signal_handler(int signum)
{
    (void)signum;
    keep_running = 0;
}

int main(void)
{
    struct gpiod_chip *chip;
    struct gpiod_line_request *request;
    struct gpiod_line_settings *settings;
    struct gpiod_line_config *line_config;
    struct gpiod_request_config *request_config;
    unsigned int offset = GPIO_OFFSET;

    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);

    chip = gpiod_chip_open(GPIO_CHIP);
    if (!chip) {
        perror("gpiod_chip_open");
        return 1;
    }

    settings = gpiod_line_settings_new();
    line_config = gpiod_line_config_new();
    request_config = gpiod_request_config_new();
    if (!settings || !line_config || !request_config) {
        fprintf(stderr, "Не удалось создать конфигурацию GPIO\n");
        gpiod_request_config_free(request_config);
        gpiod_line_config_free(line_config);
        gpiod_line_settings_free(settings);
        gpiod_chip_close(chip);
        return 1;
    }

    if (gpiod_line_settings_set_direction(
        settings, GPIOD_LINE_DIRECTION_OUTPUT) < 0 ||
        gpiod_line_settings_set_output_value(
            settings, GPIOD_LINE_VALUE_INACTIVE) < 0 ||
        gpiod_line_config_add_line_settings(
            line_config, &offset, 1, settings) < 0) {

```

```

        fprintf(stderr, "Не удалось настроить линию GPIO\n");
        gpiod_request_config_free(request_config);
        gpiod_line_config_free(line_config);
        gpiod_line_settings_free(settings);
        gpiod_chip_close(chip);
        return 1;
    }

    gpiod_request_config_set_consumer(request_config, "wavegen");
    request = gpiod_chip_request_lines(chip, request_config, line_config);
    if (!request) {
        perror("gpiod_chip_request_lines");
        gpiod_request_config_free(request_config);
        gpiod_line_config_free(line_config);
        gpiod_line_settings_free(settings);
        gpiod_chip_close(chip);
        return 1;
    }

    printf("Генератор запущен на линии %u; Ctrl+C для остановки\n", offset);
    while (keep_running) {
        /* Реализуйте генерацию меандра здесь. */
    }

    gpiod_line_request_set_value(
        request, offset, GPIOD_LINE_VALUE_INACTIVE
    );
    gpiod_line_request_release(request);
    gpiod_request_config_free(request_config);
    gpiod_line_config_free(line_config);
    gpiod_line_settings_free(settings);
    gpiod_chip_close(chip);
    return 0;
}

```

Скомпилируйте программу с библиотекой `libgpiod`:

```
gcc -Wall -Wextra -Wpedantic generator.c -o generator -lgpiod
```

Проверка

Проверка сборки. GCC должен завершиться без ошибок и предупреждений, а команда `ldd ./generator` должна показывать зависимость от `libgpiod.so.2`.

Реализация и измерение генератора

Программа должна принимать длительности высокого и низкого уровней в микросекундах, непрерывно формировать сигнал и корректно освобождать линию после Ctrl+C. Пример запуска для периода 500 мкс:

./generator 250 250

При выключенном питании подключите входной канал анализатора к PE16, а GND — к GND платы. После включения выполните захват в PulseView и измерьте длительности высокого и низкого уровней, период и частоту.

Опасность повреждения

Не соединяйте PE16, настроенный как выход, с другим активным выходом. Конфликт логических уровней может повредить оборудование.

Внимание

Пользовательский процесс Linux не обеспечивает жёсткий реальный масштаб времени. Планировщик, системные вызовы и функция ожидания вносят задержку и джиттер, поэтому измеренные интервалы могут отличаться от заданных, особенно при малых длительностях.

Проверка

Ожидаемый результат. Анализатор показывает периодический прямоугольный сигнал. После Ctrl+C программа завершается, а gpioinfo больше не показывает потребителя wavegen для выбранной линии.

Задание

1. Установите Sigrok, прошивку fx2lafw и PulseView.
2. Подключите анализатор и подтвердите его обнаружение.
3. Измерьте параметры тестового импульса длительностью 10 мс.
4. Подключитесь к TX платы и декодируйте UART 115200 8N1.
5. Установите libgpiod 2.x и пакет заголовочных файлов.
6. Подтвердите соответствие PE16 фактическому GPIO-чипу и смещению линии.
7. Дополните каркас генератора обработкой двух аргументов и формированием меандра.
8. Обеспечьте корректное завершение по Ctrl+C и освобождение GPIO-линии.
9. Соберите и запустите генератор с длительностями 250 и 250 мкс.
10. Измерьте высокий и низкий уровни, период и частоту в PulseView и сохраните захват.
11. Продемонстрируйте работу преподавателю.

Контрольные вопросы

1. Для чего нужен логический анализатор при отладке цифровых сигналов?
2. Какие параметры необходимо настроить в PulseView для декодирования UART?
3. Что такое GPIO-линия в терминах `libgpiod` 2.x и как к ней обратиться из программы?
4. Чем управление GPIO через `libgpiod` отличается от прямой записи в регистры микроконтроллера?

Требования к отчёту

Отчёт должен содержать:

- цель работы;
- модель анализатора и вывод `sigrok-cli --scan`;
- настройки и результат декодирования UART;
- версию `libgpiod` и вывод, подтверждающий выбранные GPIO-чип и смещение;
- исходный код и команду сборки генератора;
- параметры запуска и сохранённый захват PulseView;
- таблицу заданных и измеренных длительностей, периода и частоты;
- описание завершения программы и освобождения линии;
- ответы на контрольные вопросы;
- краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.